

✓ Optimizing Sentence Similarity Analysis in Medical Texts with Transformer Models

```
from sentence_transformers import SentenceTransformer
from transformers import AutoTokenizer, AutoModel
import torch
from torch.nn.functional import cosine_similarity
import os
```

✓ Working for obtaining optimal model and metric

The function below will be used for computing embeddings and in similarity methods such as cosine, jaccard, euclidean, manhattan, and pearson correlation similarity.

```
def compute_embeddings(s1, s2, model_name):
    """
    Generating embeddings for two input sentences using a specified model.

    Args:
    - s1 (str): The first sentence.
    - s2 (str): The second sentence.
    - model_name (str): The pre-trained model to use for generating embeddings.

    Returns:
    - embeddings (torch.Tensor): A torch tensor containing the embeddings for the two sentences.
    """
    # Run the device on GPU
    device = torch.device("cuda")

    if 'sentence-transformers' in model_name:
        # Load the SentenceTransformer model and send it to the device
        model = SentenceTransformer(model_name).to(device)
        # Encode the sentences and send the resulting tensors to the device
        embeddings = model.encode([s1, s2], convert_to_tensor=True).to(device)
    else:
        # Load tokenizer and model, send model to the device
        tokenizer = AutoTokenizer.from_pretrained(model_name)
        model = AutoModel.from_pretrained(model_name).to(device)

        # Prepare the inputs and send them to the device
        inputs = tokenizer([s1, s2], padding=True, return_tensors='pt', truncation=True, max_length=512).to(device)

        with torch.no_grad():
```

```

    """
    # Compute the model outputs and send them to the device
    outputs = model(**inputs)
    # Compute the mean of the last hidden state
    embeddings = outputs.last_hidden_state.mean(dim=1)

    # Return embeddings in tensor
    return embeddings

def compute_cosine_similarity(s1, s2, model_name):
    """
    Compute the cosine similarity between two sentence embeddings on GPU.

    Args:
    - s1 (str): The first sentence.
    - s2 (str): The second sentence.
    - model_name (str): The pre-trained model to use for generating embeddings.

    Returns:
    - sim_score (float): The cosine similarity score.
    """
    # Generate embeddings
    embeddings = compute_embeddings(s1, s2, model_name)

    # Compute cosine similarity using PyTorch's built-in function (expects inputs to be of shape (1, number_of_features))
    sim_score = cosine_similarity(embeddings[0].unsqueeze(0), embeddings[1].unsqueeze(0))

    # Return the similarity score as a Python float
    return sim_score.item()

```

Since the methods such as euclidean and manhattan distances calculate distances in linear form, they can't be used for similarity. But their approach can be adopted to call them as dissimilarity in linear form and thus adopted to normalize in range of 0 to 1 after converting to similarity.

There are number of functions which convert the distance to similarity including exponential decay function and sigmoid function.

1. $(1 / (1 + d))$:

◦ **Advantages:**

- Bounded between 0 and 1, which is useful for interpreting similarity.
- Simple and intuitive.
- Symmetric measure.

◦ **Considerations:**

- Linear transformation of distances.
- May not emphasize small differences as strongly as other methods.
- Suitable for general purposes and easy to implement.

2. Exponential Decay Function ($e^{(-d)}$):

◦ Advantages:

- Rapidly decreases similarity with increasing distance.
- Naturally bounded between 0 and 1.
- Useful for emphasizing small differences.

◦ Considerations:

- Sensitive to outliers or very large distances.
- May not be suitable if you want a gradual transition.

3. Sigmoid Function ($1 / (1 + e^{(-d)})$):

◦ Advantages:

- Bounded between 0 and 1.
- Non-linear transformation.
- Smooth gradation of similarity scores.

◦ Considerations:

- Versatile and widely used.
- Can handle a wide range of distances.
- Easy to interpret.

In summary:

- Use $(1 / (1 + d))$ for simplicity and direct interpretation.
- Use **exponential decay** if you want to penalize larger distances more heavily.
- Use **sigmoid** for versatility and smooth gradation.

Source: Conversation with Bing, 4/12/2024

After detail search and finding the best suitable conversion functions, I decided to use the following formula since it is simple and easy to interpret. In our cases, the distances won't be more large. In addition, It has non-linear nature.

$$\frac{1}{1 + d(s1, s2)}$$

```
def distance_to_similarity(dist_score):
    # Apply the exponential decay function to convert distance to similarity
    # sim_score = torch.exp(-dist_score)

    # Apply simple and fast function to convert distance to similarity
    sim_score = 1 / (1 + dist_score)

    # Apply sigmoid function to convert distance to similarity
    # sim_score = 1 / (1 + torch.exp(-dist_score))
    return sim_score.item()

def compute_euclidean_similarity(s1, s2, model_name):
    """
    Compute the Euclidean distance between two sentence embeddings on GPU.

    Args:
    - s1 (str): The first sentence.
    - s2 (str): The second sentence.
    - model_name (str): The pre-trained model to use for generating embeddings.

    Returns:
    - dist_score (float): The Euclidean distance score.
    """
    # Generate embeddings
    embeddings = compute_embeddings(s1, s2, model_name)

    # Compute Euclidean distance using PyTorch's operations
    dist_score = torch.norm(embeddings[0] - embeddings[1], p=2)

    # Return the distance score as a Python float
    return distance_to_similarity(dist_score)
```

```
def compute_manhattan_similarity(s1, s2, model_name):
    """
    Compute the Manhattan distance between two sentence embeddings on GPU.

    Args:
    - s1 (str): The first sentence.
    - s2 (str): The second sentence.
    - model_name (str): The pre-trained model to use for generating embeddings.

    Returns:
    - dist_score (float): The Manhattan distance score.
    """
    # Generate embeddings
    embeddings = compute_embeddings(s1, s2, model_name)

    # Compute Manhattan distance using PyTorch's operations
    dist_score = torch.sum(torch.abs(embeddings[0] - embeddings[1]))

    # Return the distance score as a Python float
    return distance_to_similarity(dist_score)
```

The following function computes the pearson correlation and converts the score into similarity score. This task is accomplished by using a function called Half-Shift Method.

Half-Shift Function: This method converts the correlation into similarity score by adding 1 to the correlation coefficient and then dividing by 2 to shift the range. It maintains the directionality of the relationship (positive or negative) while converting it into a non-negative similarity score.

1. **Disadvantage:** A strong negative correlation close to -1 will result in a similarity score close to 0, which could be interpreted as "not similar" rather than "oppositely similar".
2. **Advantage:** This method maintains the direction of the relationship which is important linearly transforming the correlation while preserving the direction, instead of the strength of the relationship regardless of the direction. This helps us in calculating the similarity at best possible.

The half-shift method for converting a Pearson correlation coefficient to a similarity score is given by the following equation:

$$\text{Similarity} = \frac{\text{Correlation} + 1}{2}$$

```
def compute_pearson_correlation(s1, s2, model_name):
    """
    Compute the Pearson correlation coefficient between two sentence embeddings on GPU
    and convert it to a similarity score ranging from 0 to 1 using the half-shift method.

    Args:
    - s1 (str): The first sentence.
    - s2 (str): The second sentence.
    - model_name (str): The pre-trained model to use for generating embeddings.

    Returns:
    - similarity_score (float): The similarity score ranging from 0 to 1.
    """
    # Generate embeddings
    embeddings = compute_embeddings(s1, s2, model_name)

    # Stack the embeddings to create a 2D tensor
    stacked_embeddings = torch.stack((embeddings[0], embeddings[1]))

    # Compute the Pearson correlation matrix and extract the correlation coefficient
    corr_matrix = torch.corrcoef(stacked_embeddings)
    corr = corr_matrix[0, 1].item() # Extract the correlation coefficient

    # Convert the correlation coefficient to a similarity score using the half-shift method
    similarity_score = (corr + 1) / 2

    # Return the Half-Shift similarity score
    return similarity_score

# Test function
similarity_score = compute_pearson_correlation("I had a bad day", "Everything was terrible today", 'sentence-transformers/all-MiniLM-L6-v2')
print(f"Pearson Correlation-based Similarity Score: {similarity_score}")

    Pearson Correlation-based Similarity Score: 0.828032374382019
```

```
def compute_jaccard_similarity(s1, s2, model_name, threshold=0.00):
    """
    Compute the Jaccard similarity between two sentence embeddings on GPU.

    Args:
    - s1 (str): The first sentence.
    - s2 (str): The second sentence.
    - model_name (str): The pre-trained model to use for generating embeddings.
    - threshold (float): The threshold to determine feature presence.

    Returns:
    - sim_score (float): The Jaccard similarity score.
    """
    # Generate embeddings
```

```
embeddings = compute_embeddings(s1, s2, model_name)

# Determine feature presence based on the threshold
binary_vec1 = (embeddings[0] > threshold).int()
binary_vec2 = (embeddings[1] > threshold).int()

# Compute intersection and union using logical operations
intersection = torch.logical_and(binary_vec1, binary_vec2)
union = torch.logical_or(binary_vec1, binary_vec2)

# Calculate Jaccard similarity score
sim_score = intersection.sum().float() / union.sum().float()

# Return the similarity score as a Python Float
return sim_score.item()

# Test function
sim_score = compute_jaccard_similarity("I had a bad day", "Everything was terrible today", 'sentence-transformers/all-MiniLM-L6-v2')
print(f"Jaccard Similarity Score: {sim_score}")

Jaccard Similarity Score: 0.5559999942779541
```

✓ Character-Based Similarity Analysis

I have used the following methods which do not require the models, and are suitable particularly in the context of NLP and biomedical informatics:

1. **Qgram [0-1]:** This is a string similarity measure that compares two strings based on the number of q-grams (substrings of length 'q') they share. A score close to 1 indicates high similarity, while a score close to 0 indicates low similarity.
2. **Wordnet [0-1]:** WordNet is a lexical database for the English language that helps in semantic similarity measurement. It groups English words into sets of synonyms called synsets, provides short definitions and usage examples, and records the various semantic relations between these synonym sets.

```
import nltk
from nltk.util import ngrams

def compute_qgram_similarity(s1, s2, q=2):
    """
    Compute the Qgram similarity between two sentences.

    Args:
    - s1 (str): The first sentence.
    - s2 (str): The second sentence.
    - q (int): The length of each q-gram.

    Returns:
    - sim_score (float): The Qgram similarity score.
    """
    # Tokenize the sentences and generate q-grams
    qgrams1 = set(ngrams(nltk.word_tokenize(s1), q))
    qgrams2 = set(ngrams(nltk.word_tokenize(s2), q))

    # Calculate the intersection and union
    intersection = qgrams1.intersection(qgrams2)
    union = qgrams1.union(qgrams2)

    # Calculate the Qgram similarity score
    sim_score = len(intersection) / len(union)

    # Return the Qgram similarity score
    return sim_score
```

The function uses WordNet from the NLTK library to assess how similar two sentences are. It breaks down sentences into words, finds synonyms for each, and calculates a similarity score. This score reflects how semantically close the sentences are, which is useful for various language processing tasks. The function's effectiveness depends on the similarity measure chosen, as it influences the interpretation of semantic relationships.


```

from nltk.corpus import wordnet as wn

def compute_wordnet_similarity(s1, s2, measure):
    """
    Compute the semantic similarity between two sentences using WordNet.

    This function tokenizes both sentences and retrieves the synsets for each word.
    It then flattens the list of synsets and calculates the similarity score for each
    pair of synsets that share the same part of speech (POS). The average of all
    non-None scores is calculated and returned as the final similarity score.

    Parameters:
    s1 (str): The first sentence.
    s2 (str): The second sentence.
    measure (str): The similarity measure to use ('path' or 'wup').

    Returns:
    float: The average semantic similarity score between the two sentences.
    """

    # Tokenize and get the synsets for each word in the sentences
    synsets1 = [wn.synsets(word) for word in nltk.word_tokenize(s1)]
    synsets2 = [wn.synsets(word) for word in nltk.word_tokenize(s2)]

    # Flatten the list of synsets
    synsets1 = [item for sublist in synsets1 for item in sublist]
    synsets2 = [item for sublist in synsets2 for item in sublist]

    # Calculate the similarity score for each pair of synsets using the specified measure
    scores = []
    for synset1 in synsets1:
        for synset2 in synsets2:
            if synset1.pos() == synset2.pos(): # Ensure the synsets have the same POS
                if measure == 'path':
                    score = synset1.path_similarity(synset2)
                elif measure == 'wup':
                    score = synset1.wup_similarity(synset2)
                # Add other measures here
                if score is not None:
                    scores.append(score)

    # Average the scores
    sim_score = sum(scores) / len(scores) if scores else 0

    return sim_score

# Test Function
sentence1 = "Dogs are great pets."
sentence2 = "Cats are good animals."

```

```
# Compare different WordNet similarity measures
for measure in ['path', 'wup']:
    score = compute_wordnet_similarity(sentence1, sentence2, measure)
    print(f"WordNet similarity using {measure}: {score}")
```

```
WordNet similarity using path: 0.2241912649397844
WordNet similarity using wup: 0.39377043547918644
```

✓ Dataset for Performance:

I utilized the BIOSSES dataset, which is a benchmark dataset for biomedical semantic similarity estimation. The data was collected by having experts annotate sentence pairs from biomedical literature. For each sentence pair, multiple persons provided their judgments on the semantic similarity. Specifically, five different persons rated each pair, and the final score column in the dataset represents the mean of these ratings. This approach ensures a more reliable and nuanced understanding of the semantic relationships within the biomedical context.

"The Pearson correlation between the gold standard scores and the scores estimated by the models was used as the evaluation metric. The strength of correlation can be assessed by the general guideline proposed by Evans (1996) as follows:

- **very strong:** 0.80–1.00
- **strong:** 0.60–0.79
- **moderate:** 0.40–0.59
- **weak:** 0.20–0.39
- **very weak:** 0.00–0.19"

References:

Soğancıoğlu, Gizem, Hakime Öztürk, and Arzucan Özgür. "BIOSSES: a semantic sentence similarity estimation system for the biomedical domain." *Bioinformatics* 33.14 (2017): i49-i58.

<https://huggingface.co/datasets/qanastek/Biosses-BLUE>

```
import pandas as pd
from datasets import load_dataset
```

```
# Load the BIOSSES dataset
biosses_dataset = load_dataset("biosses")
```

```
# Although this is called as train, but it has no test set, meaning that the train set is the original dataset and full.
biosses_df = pd.DataFrame(biosses_dataset['train'])
biosses_df.head()
```

	sentence1	sentence2	score
0	Here, looking for agents that could specifical...	Not surprisingly, GATA2 knockdown in KRAS muta...	2.2
1	MLL-FKBP and MLL-AF9 transformed cells showed ...	Regardless of the mechanism for transcriptiona...	3.2
2	The oncogenic activity of mutant Kras appears ...	Oncogenic KRAS mutations are common in cancer.	2.0
3	Consequently miRNAs have been demonstrated to ...	Given the extensive involvement of miRNA in ph...	2.8
4	We then sought to reassess the regulation of m...	Importantly, our reassessment revealed that th...	2.4

It is essential to normalize the dataset to scale of 0 - 1 to represent the true similarity.

```
# Normalize the score to scale of 0.0 to 1.0 representing true similarity
biosses_df['norm_score'] = biosses_df['score'] / 4.0
```

```
biosses_df.head()
```

	sentence1	sentence2	score	norm_score
0	Here, looking for agents that could specifical...	Not surprisingly, GATA2 knockdown in KRAS muta...	2.2	0.55
1	MLL-FKBP and MLL-AF9 transformed cells showed ...	Regardless of the mechanism for transcriptiona...	3.2	0.80
2	The oncogenic activity of mutant Kras appears ...	Oncogenic KRAS mutations are common in cancer.	2.0	0.50
3	Consequently miRNAs have been demonstrated to ...	Given the extensive involvement of miRNA in ph...	2.8	0.70
4	We then sought to reassess the regulation of m...	Importantly, our reassessment revealed that th...	2.4	0.60

✓ Approach:

The approach is to predict the models and scores function on dataset. then identify the best one combination with the help of hyper parameter tuning. In the end, the final function will be composed of the best metric score and best model. In case of character-based similarity such as Q-gram and wordnet, of course model will not be used.

```

from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.model_selection import GridSearchCV
import pandas as pd

# Custom estimator class for hyperparameter tuning
class SimilarityEstimator(BaseEstimator, TransformerMixin):
    def __init__(self, model_name='sentence-transformers/all-distilroberta-v1', similarity_function=compute_cosine_similarity):
        self.model_name = model_name
        self.similarity_function = similarity_function

    def fit(self, X, y=None):
        # No fitting necessary for similarity functions
        return self

    def predict(self, X):
        # Compute similarity scores for each pair of sentences
        scores = [self.similarity_function(s1, s2, self.model_name) for s1, s2 in X]
        return scores

# Define the parameter grid to search over
param_grid = {
    'model_name': [
        'sentence-transformers/all-mpnet-base-v2',
        'sentence-transformers/all-MiniLM-L6-v2',
        'sentence-transformers/all-distilroberta-v1'
    ],
    'similarity_function': [
        compute_cosine_similarity,
        compute_euclidean_similarity,
        compute_manhattan_similarity,
        compute_pearson_correlation,
        compute_jaccard_similarity,
    ]
}

from sklearn.metrics import make_scorer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

def cosine_scorer(y_true, y_pred):
    # Reshape the vectors to 2D arrays for cosine_similarity function
    y_pred_2d = np.array(y_pred).reshape(1, -1)
    y_true_2d = np.array(y_true).reshape(1, -1)
    # Calculate the cosine similarity
    cos_sim = cosine_similarity(y_pred_2d, y_true_2d)
    return cos_sim[0][0]

```

```
# biosses_df = pd.read_csv('your_dataset.csv')
X = biosses_df[['sentence1', 'sentence2']].values
y = biosses_df['norm_score'].values

# Create the scorer object
cosine_scorer = make_scorer(cosine_scorer)

# Use cosine_scorer as the scoring parameter in GridSearchCV
grid_search = GridSearchCV(estimator=SimilarityEstimator(), param_grid=param_grid, scoring=cosine_scorer, cv=5)

# Run the grid search on your data
grid_search.fit(X, y)

# Note: The best score at the end was printed with negative sign (mistake).
```

```

modules.json: 100% 349/349 [00:00<00:00, 16.9kB/s]
config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 6.94kB/s]
README.md: 100% 10.7k/10.7k [00:00<00:00, 469kB/s]
sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 3.39kB/s]
config.json: 100% 612/612 [00:00<00:00, 38.1kB/s]
model.safetensors: 100% 90.9M/90.9M [00:00<00:00, 185MB/s]
tokenizer_config.json: 100% 350/350 [00:00<00:00, 17.9kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 4.85MB/s]
tokenizer.json: 100% 466k/466k [00:00<00:00, 4.87MB/s]
special_tokens_map.json: 100% 112/112 [00:00<00:00, 3.83kB/s]
1_Pooling/config.json: 100% 190/190 [00:00<00:00, 8.63kB/s]
modules.json: 100% 349/349 [00:00<00:00, 12.1kB/s]
config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 6.88kB/s]
README.md: 100% 10.3k/10.3k [00:00<00:00, 449kB/s]
sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 2.25kB/s]
config.json: 100% 653/653 [00:00<00:00, 45.4kB/s]
model.safetensors: 100% 328M/328M [00:02<00:00, 145MB/s]
tokenizer_config.json: 100% 333/333 [00:00<00:00, 13.7kB/s]
vocab.json: 100% 798k/798k [00:00<00:00, 4.73MB/s]
merges.txt: 100% 456k/456k [00:00<00:00, 9.17MB/s]
tokenizer.json: 100% 1.36M/1.36M [00:00<00:00, 10.3MB/s]
special_tokens_map.json: 100% 239/239 [00:00<00:00, 13.9kB/s]
1_Pooling/config.json: 100% 190/190 [00:00<00:00, 12.2kB/s]
Best parameters found: {'model_name': 'sentence-transformers/all-distilroberta-v1', 'similarity_function': <function c
Best score found: -0.9676242927965492

```

```

# Print the best parameters and the corresponding score
print("Best parameters found: ", grid_search.best_params_)
print("Best score found: ", grid_search.best_score_)

```

```

Best parameters found: {'model_name': 'sentence-transformers/all-distilroberta-v1', 'similarity_function': <function compute_cosine_similarity at 0x7f3f
Best score found: 0.9676242927965492

```

✓ Hyperparameter Tuning using the WordNet and Qgram methods

```

class SimilarityEstimatorNonModel(BaseEstimator, TransformerMixin):
    def __init__(self, q=None, measure=None):
        self.q = q
        self.measure = measure

    def fit(self, X, y=None):
        # No fitting is necessary for this estimator
        return self

    def predict(self, X):
        # Compute the similarity scores
        return compute_similarity(X, q=self.q, measure=self.measure)

    def get_params(self, deep=True):
        # Return the parameters
        return {'q': self.q, 'measure': self.measure}

    def set_params(self, **parameters):
        # Set the parameters
        for parameter, value in parameters.items():
            setattr(self, parameter, value)
        return self

def custom_scorer(y_true, y_pred):
    # Note: y_pred is a list of similarity scores
    y_pred_2d = np.array(y_pred).reshape(1, -1)
    y_true_2d = np.array(y_true).reshape(1, -1)
    # Calculate the cosine similarity
    cos_sim = cosine_similarity(y_pred_2d, y_true_2d)
    return cos_sim[0][0]

```

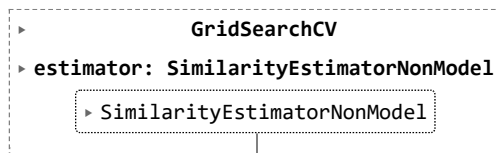
```

cosine_scorer = make_scorer(custom_scorer)

# Define the sentences and normalized scores in X and y respectively
X = biosses_df[['sentence1', 'sentence2']].values
y = biosses_df['norm_score'].values

# Run the grid search
grid_search.fit(X, y)

```



```

# Print the best parameters and the corresponding score
print("Best parameters found: ", grid_search.best_params_)
print("Best score found: ", grid_search.best_score_)

```

```

Best parameters found: {'measure': 'path', 'q': 2}
Best score found: 0.6922701008645189

```

✓ Conclusion:

Since the model `all-distilroberta-v1` has performed better among all with `cosine_similarity` function. Therefore, I will make a function utilizing the model using the similarity function in the same.

```

from sentence_transformers import SentenceTransformer
from scipy.spatial.distance import cosine
import torch

```

```

def compute_similarity(s1, s2):
    """

```